TC-HKEM: A Triple-Committed Hybrid KEM for Post-Quantum Secrets Management

Ciphertext-Bound, Passphrase-Committed Key Derivation
with Provable Best-of-Both-Worlds Security

Zack Fitch
Defense of the Artifacts Project
`github.com/johnzfitch/dota`

March 2026

Abstract

We present TC-HKEM (Triple-Committed Hybrid KEM), a key derivation construction for password-protected secret vaults that achieves three security properties simultaneously: (1) IND-CCA security if *either* ML-KEM-768 or X25519 is secure (best-of-both-worlds, via ciphertext binding per Giacon–Heuer–Poettering 2018), (2) passphrase binding that prevents decryption even given both asymmetric private keys without the Argon2-derived master key, and (3) complete domain separation across vault format versions. TC-HKEM is implemented in DOTA v7, a Rust-based post-quantum secrets manager. We provide full security proofs in the random oracle model and trace the mathematical evolution from the X25519-only v1 construction through to the triple-committed v7 combiner.

1 Introduction

The harvest-now-decrypt-later threat model motivates hybrid post-quantum/classical encryption for data at rest. A vault that stores secrets encrypted under ML-KEM-768 [2] alongside classical X25519 [3] must combine the two shared secrets into a single symmetric key. The security of this combination depends critically on the *combiner* construction.

The gap in dota v6. The predecessor construction derives per-secret AES-256 keys as:

$$K_{\text{v6}} = \mathsf{HKDF}(\underbrace{\mathsf{ss}_{\mathsf{KEM}} \,\|\, \mathsf{ss}_{\mathsf{DH}}}_{\text{64 bytes}},\ \texttt{"dota-v6-hkdf-salt"},\ \texttt{"dota-v6-secret"}) \tag{1}$$

This omits the KEM ciphertext $\mathsf{ct}_{\mathsf{KEM}}$ and ephemeral public key $\mathsf{eph}_{\mathsf{pk}}$ from the HKDF input. Per the GHP18 combiner theorem [1], without ciphertext binding the security reduction requires *both* components to be IND-CCA (worst-of-both-worlds).

TC-HKEM (this work). We construct a combiner that includes ciphertext binding and a novel passphrase commitment:

$$\boxed{\tau = \mathsf{HMAC}(\mathsf{mk},\ \mathsf{ct}_{\mathsf{KEM}} \,\|\, \mathsf{eph}_{\mathsf{pk}}) \qquad K = \mathsf{HKDF}(\mathsf{ss}_{\mathsf{KEM}} \,\|\, \mathsf{ss}_{\mathsf{DH}} \,\|\, \mathsf{ct}_{\mathsf{KEM}} \,\|\, \mathsf{eph}_{\mathsf{pk}} \,\|\, \tau)} \tag{2}$$

where mk is the Argon2id-derived master key and τ is a 32-byte passphrase commitment tag. The total IKM is 1216 bytes for ML-KEM-768.

2 Preliminaries

2.1 Notation

We write $x \xleftarrow{\$} S$ for uniform sampling from set S, $\|$ for byte concatenation, and $|x|$ for byte length. All security parameters are implicit at $\lambda = 256$.

Definition 2.1 (Key Encapsulation Mechanism)**.** *A KEM* $\Pi = (\mathsf{KeyGen}, \mathsf{Encaps}, \mathsf{Decaps})$ *consists of:*

- $\mathsf{KeyGen}(1^\lambda) \to (ek, dk)$*: key generation*
- $\mathsf{Encaps}(ek) \to (\mathsf{ss}, \mathsf{ct})$*: encapsulation producing shared secret and ciphertext*
- $\mathsf{Decaps}(dk, \mathsf{ct}) \to \mathsf{ss}$*: decapsulation recovering the shared secret*

Π *is* IND-CCA secure *if no efficient adversary can distinguish the real shared secret from random, even with access to a decapsulation oracle.*

Definition 2.2 (Diffie-Hellman Group)**.** *A DH group* $\Gamma = (\mathsf{KeyGen}_{\mathsf{DH}}, \mathsf{DH})$ *consists of key generation and a Diffie-Hellman function.* Γ *satisfies the* gap-CDH *assumption if CDH is hard given a DDH oracle.*

Definition 2.3 (PRF Security)**.** *A family* $F_k : \{0,1\}^* \to \{0,1\}^{32}$ *is a secure PRF if no efficient adversary can distinguish* $F_k(\cdot)$ *from a truly random function, given oracle access.*

2.2 Cryptographic Building Blocks

Component	Instantiation	Reference
Π_{KEM}	ML-KEM-768 (FIPS 203)	[2]
Γ_{DH}	X25519 (Curve25519)	[3]
F	HMAC-SHA256	[4]
HKDF	HKDF-SHA256	[5]
AES-GCM	AES-256-GCM	[6]
KSF	Argon2id (64 MiB, t=3, p=4)	[7]

3 The TC-HKEM Construction

Definition 3.1 (TC-HKEM)**.** *A* Triple-Committed Hybrid KEM *is a tuple* $(\mathsf{Setup}, \mathsf{Encaps}_{\mathrm{TC}}, \mathsf{Decaps}_{\mathrm{TC}})$ *parameterized by a KEM* Π_{KEM}*, a DH group* Γ_{DH}*, a PRF family* F*, and a key derivation function* HKDF *(modeled as a random oracle):*

Setup(1^λ)*:*

$$(ek, dk) \leftarrow \Pi_{\mathsf{KEM}}.\mathsf{KeyGen}(1^\lambda) \tag{3}$$

$$(pk_{\mathsf{DH}}, sk_{\mathsf{DH}}) \leftarrow \Gamma_{\mathsf{DH}}.\mathsf{KeyGen}(1^\lambda) \tag{4}$$

$$\mathsf{mk} \leftarrow \mathrm{Argon2id}(\mathsf{passphrase}, \mathsf{salt}) \tag{5}$$

Encaps$_{\mathrm{TC}}(ek, pk_{\mathsf{DH}}, \mathsf{mk})$*:*

1. $(\mathsf{ss}_{\mathsf{KEM}}, \mathsf{ct}_{\mathsf{KEM}}) \leftarrow \Pi_{\mathsf{KEM}}.\mathsf{Encaps}(ek)$
2. $(\mathsf{eph}_{\mathsf{pk}}, \mathsf{eph}_{\mathsf{sk}}) \leftarrow \Gamma_{\mathsf{DH}}.\mathsf{KeyGen}()$
3. $\mathsf{ss}_{\mathsf{DH}} \leftarrow \Gamma_{\mathsf{DH}}.\mathsf{DH}(\mathsf{eph}_{\mathsf{sk}}, pk_{\mathsf{DH}})$
4. $\tau \leftarrow F_{\mathsf{mk}}(\mathsf{ct}_{\mathsf{KEM}} \| \mathsf{eph}_{\mathsf{pk}})$ passphrase commitment
5. $K \leftarrow \mathsf{HKDF}(\mathsf{ss}_{\mathsf{KEM}} \| \mathsf{ss}_{\mathsf{DH}} \| \mathsf{ct}_{\mathsf{KEM}} \| \mathsf{eph}_{\mathsf{pk}} \| \tau)$ ciphertext-bound derivation

Output $(K, \mathsf{ct}_{\mathsf{KEM}}, \mathsf{eph}_{\mathsf{pk}})$.

Decaps$_{\mathrm{TC}}(dk, sk_{\mathsf{DH}}, \mathsf{mk}, \mathsf{ct}_{\mathsf{KEM}}, \mathsf{eph}_{\mathsf{pk}})$*: identical to steps 1–5 with* Decaps *replacing* Encaps *and* $\mathsf{DH}(sk_{\mathsf{DH}}, \mathsf{eph}_{\mathsf{pk}})$ *replacing the ephemeral DH.*

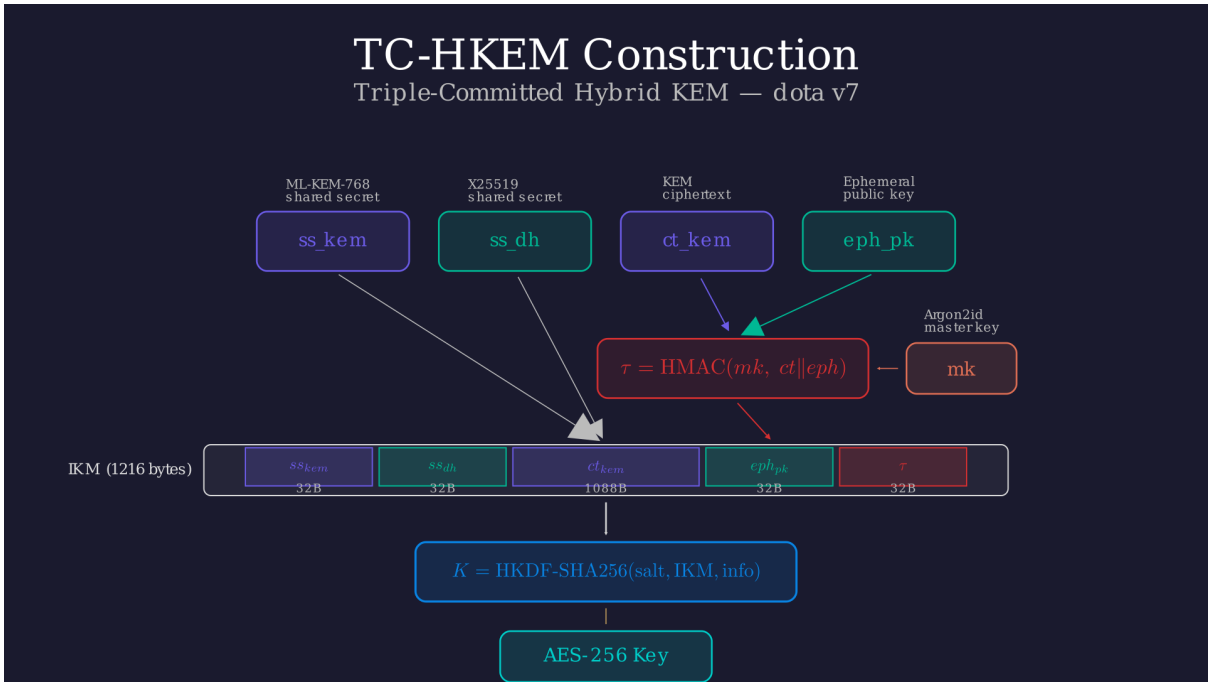

Figure 1: TC-HKEM key derivation pipeline. The 1216-byte IKM includes shared secrets ($\mathsf{ss}_{\mathsf{KEM}}$, $\mathsf{ss}_{\mathsf{DH}}$), ciphertexts ($\mathsf{ct}_{\mathsf{KEM}}$, $\mathsf{eph}_{\mathsf{pk}}$), and the passphrase commitment τ.

3.1 Concrete Parameters

IKM Component	Bytes	Source
$\mathsf{ss}_{\mathsf{KEM}}$	32	ML-KEM-768 shared secret
$\mathsf{ss}_{\mathsf{DH}}$	32	X25519 shared secret
$\mathsf{ct}_{\mathsf{KEM}}$	1088	ML-KEM-768 ciphertext
$\mathsf{eph}_{\mathsf{pk}}$	32	X25519 ephemeral public key
τ	32	HMAC-SHA256 commitment
Total IKM	**1216**	

Domain separation labels:

$$\mathsf{salt}_{\mathrm{v7}} = \texttt{"dota-v7-tchkem-salt"} \tag{6}$$

$$\mathsf{info}_{\mathrm{v7}} = \texttt{"dota-v7-secret-key"} \tag{7}$$

4 Security Analysis

4.1 Theorem 1: Best-of-Both-Worlds IND-CCA

Theorem 4.1 (IND-CCA Security)**.** *Let $\mathcal{A}$ be an adversary against TC-HKEM making at most q_D decapsulation queries and q_H random oracle queries. Then:*

$$\mathsf{Adv}^{\text{ind-cca}}_{\text{TC-HKEM}}(\mathcal{A}) \;\leq\; \mathsf{Adv}^{\text{ind-cca}}_{\Pi_{\mathsf{KEM}}}(\mathcal{B}_1) \;+\; \mathsf{Adv}^{\text{gap-cdh}}_{\Gamma_{\mathsf{DH}}}(\mathcal{B}_2) \;+\; \frac{q_H}{2^{256}} \tag{8}$$

where $\mathcal{B}_1, \mathcal{B}_2$ are efficient reductions. In particular, TC-HKEM is IND-CCA secure if ***either*** *Π_{KEM} or Γ_{DH} is secure.*

Proof. We proceed by a sequence of games.

Figure 2: Game-hopping sequence for Theorem 4.1. Each transition preserves computational indistinguishability, reducing to the security of a single component.

Game 0 (Real). The standard IND-CCA experiment. The challenger runs $\mathsf{Encaps}_{\mathrm{TC}}$ to produce challenge $(K_0^*, \mathsf{ct}^*_{\mathsf{KEM}}, \mathsf{eph}^*_{\mathsf{pk}})$. $K_1^* \xleftarrow{\$} \{0,1\}^{256}$. The adversary receives $(K_b^*, \mathsf{ct}^*_{\mathsf{KEM}}, \mathsf{eph}^*_{\mathsf{pk}})$ and must guess b.

Game 1 (Random Oracle). Replace HKDF with a random oracle H. Since we are already in the ROM:

$$|\Pr[\mathsf{G}_0] - \Pr[\mathsf{G}_1]| = 0$$

Game 2 (KEM hop). Replace $\mathsf{ss}^*_{\mathsf{KEM}}$ in the challenge with $\widetilde{\mathsf{ss}}_{\mathsf{KEM}} \xleftarrow{\$} \{0,1\}^{256}$. The reduction $\mathcal{B}_1$ embeds an IND-CCA challenger for Π_{KEM} and uses its decapsulation oracle to answer $\mathcal{A}$'s queries. Crucially, $\mathsf{ct}^*_{\mathsf{KEM}}$ is included in the HKDF input (*ciphertext binding*), so $\mathcal{B}_1$ can detect and reject decapsulation queries on the challenge ciphertext. Without ciphertext binding, this step fails.

$$|\Pr[\mathsf{G}_1] - \Pr[\mathsf{G}_2]| \;\le\; \mathsf{Adv}^{\mathrm{ind\text{-}cca}}_{\Pi_{\mathsf{KEM}}}(\mathcal{B}_1)$$

Game 3 (DH hop). Replace $\mathsf{ss}^*_{\mathsf{DH}}$ with $\widetilde{\mathsf{ss}}_{\mathsf{DH}} \xleftarrow{\$} \{0,1\}^{256}$. The reduction $\mathcal{B}_2$ solves gap-CDH in Curve25519:

$$|\Pr[\mathsf{G}_2] - \Pr[\mathsf{G}_3]| \;\le\; \mathsf{Adv}^{\mathrm{gap\text{-}cdh}}_{\Gamma_{\mathsf{DH}}}(\mathcal{B}_2)$$

Game 4 (Random key). In Game 3, the random oracle input contains $\widetilde{\mathsf{ss}}_{\mathsf{KEM}}$ at a position uniquely determined by $\mathsf{ct}^*_{\mathsf{KEM}}$. Since $\widetilde{\mathsf{ss}}_{\mathsf{KEM}}$ is uniform and independent of all other values visible to $\mathcal{A}$, the output $H(\cdots)$ is uniform unless $\mathcal{A}$ queries H on the exact input:

$$\left|\Pr[\mathsf{G}_3] - \tfrac{1}{2}\right| \;\le\; \frac{q_H}{2^{256}}$$

Combining all transitions yields Equation (8). $\square$

Remark 4.2 (Why ciphertext binding is essential)**.** *In Game 2, the reduction $\mathcal{B}_1$ receives a challenge ciphertext ct^* from the IND-CCA challenger and must embed it as $\mathsf{ct}^*_{\mathsf{KEM}}$ in the TC-HKEM challenge. When $\mathcal{A}$ makes a decapsulation query on $(\mathsf{ct}', \mathsf{eph}')$ where $\mathsf{ct}' = \mathsf{ct}^*$, $\mathcal{B}_1$ must detect this and abort (since it cannot query its own oracle on the challenge). Because $\mathsf{ct}_{\mathsf{KEM}}$ is part of the HKDF input, changing any bit of ct changes the key K, making the detection straightforward. In the v6 combiner (Equation 1), $\mathsf{ct}_{\mathsf{KEM}}$ is absent from the hash input, so $\mathcal{B}_1$ cannot distinguish challenge queries from legitimate ones — the reduction fails.*

4.2 Theorem 2: Passphrase Binding

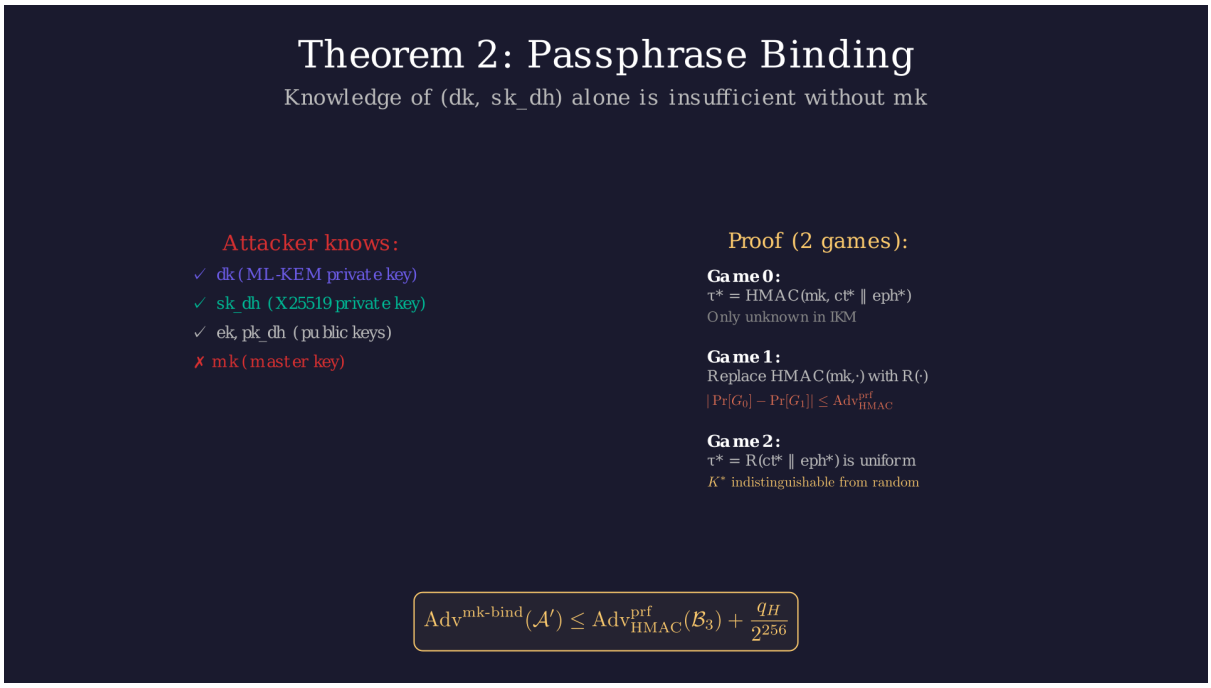

Figure 3: Passphrase binding proof structure. An attacker with both private keys but without mk cannot recover per-secret AES keys.

Theorem 4.3 (Passphrase Binding)**.** *For any adversary $\mathcal{A}'$ that knows (dk, sk_{DH}) but not mk:*

$$\mathsf{Adv}^{\text{mk-bind}}_{\text{TC-HKEM}}(\mathcal{A}') \;\leq\; \mathsf{Adv}^{\text{prf}}_{F}(\mathcal{B}_3) \;+\; \frac{q_H}{2^{256}} \tag{9}$$

That is, knowledge of both private keys is insufficient to recover K without the master key.

Proof. **Game 0.** $\mathcal{A}'$ receives $(dk, sk_{\mathsf{DH}}, ek, pk_{\mathsf{DH}})$ and challenge $(K^*_b, \mathsf{ct}^*_{\mathsf{KEM}}, \mathsf{eph}^*_{\mathsf{pk}})$. It can compute $\mathsf{ss}^*_{\mathsf{KEM}} = \mathsf{Decaps}(dk, \mathsf{ct}^*_{\mathsf{KEM}})$ and $\mathsf{ss}^*_{\mathsf{DH}} = \mathsf{DH}(sk_{\mathsf{DH}}, \mathsf{eph}^*_{\mathsf{pk}})$ directly. The only unknown in the HKDF input is:

$$\tau^* = F_{\mathsf{mk}}(\mathsf{ct}^*_{\mathsf{KEM}} \,\|\, \mathsf{eph}^*_{\mathsf{pk}})$$

Game 1. Replace F_{mk} with a truly random function $R : \{0,1\}^* \to \{0,1\}^{256}$:

$$|\Pr[\mathsf{G}_0] - \Pr[\mathsf{G}_1]| \;\leq\; \mathsf{Adv}^{\text{prf}}_{F}(\mathcal{B}_3)$$

Game 2. $\tau^* = R(\mathsf{ct}^*_{\mathsf{KEM}} \| \mathsf{eph}^*_{\mathsf{pk}})$ is uniformly random and independent of everything $\mathcal{A}'$ knows. The HKDF input contains a fresh random 32-byte value, so K^* is indistinguishable from random:

$$\left|\Pr[\mathsf{G}_1] - \tfrac{1}{2}\right| \;\leq\; \frac{q_H}{2^{256}}$$

□

Corollary 4.4 (Defense in Depth)**.** *Even if an attacker extracts both dk and* sk_{DH} *via memory dump or side channel, per-secret AES keys remain protected as long as the Argon2id-derived master key* mk *has been zeroized. This is a property that the v6 combiner does* ***not*** *possess.*

5 Vault Version Evolution

We trace the mathematical structure of each vault version, working backwards from the TC-HKEM construction in v7.

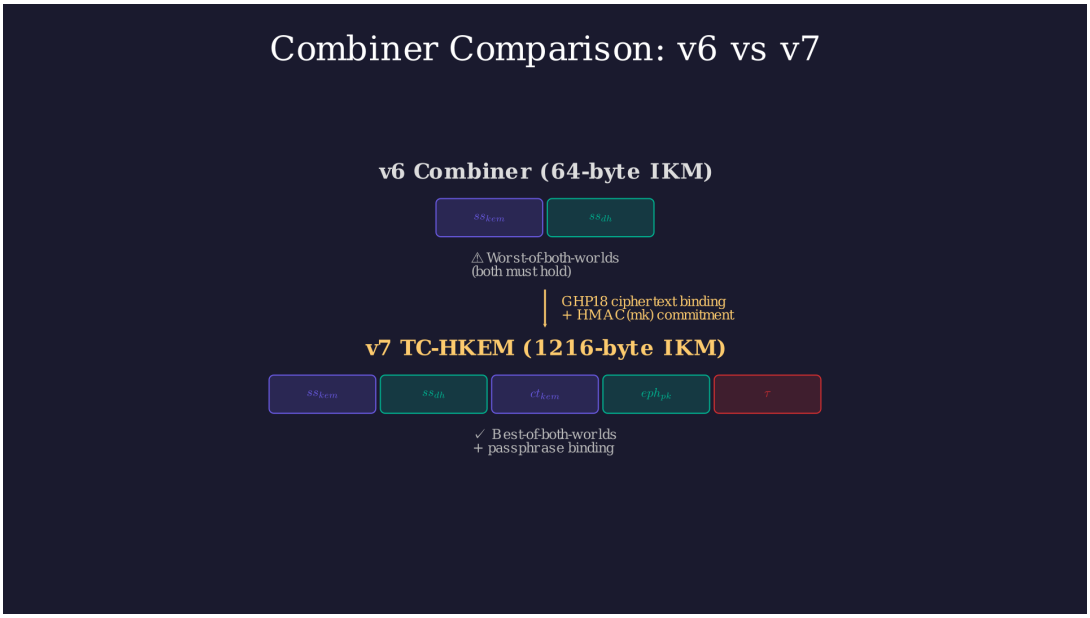

Figure 4: Combiner comparison: v6 (64-byte IKM, worst-of-both-worlds) vs. v7 TC-HKEM (1216-byte IKM, best-of-both-worlds + passphrase binding).

5.1 v7: TC-HKEM (This Work)

$$\mathsf{IKM}_{\mathrm{v7}} = \mathsf{ss}_{\mathsf{KEM}} \,\|\, \mathsf{ss}_{\mathsf{DH}} \,\|\, \mathsf{ct}_{\mathsf{KEM}} \,\|\, \mathsf{eph}_{\mathsf{pk}} \,\|\, \underbrace{\mathsf{HMAC}(\mathsf{mk}, \mathsf{ct}_{\mathsf{KEM}} \| \mathsf{eph}_{\mathsf{pk}})}_{\tau} \tag{10}$$

Properties: Best-of-both-worlds IND-CCA (Theorem 4.1), passphrase binding (Theorem 4.3), 1216-byte IKM.

5.2 v6: FIPS 203 ML-KEM Cutover

$$\mathsf{IKM}_{\mathrm{v6}} = \mathsf{ss}_{\mathsf{KEM}} \,\|\, \mathsf{ss}_{\mathsf{DH}} \tag{11}$$

Upgrade from v5: Replaced pre-FIPS Kyber768 (`pqcrypto-kyber`) with real ML-KEM-768 (`ml-kem` crate, FIPS 203 final). Added authenticated suite metadata, explicit algorithm labeling, and HMAC-SHA256 key commitment over the canonical header.

Gap: No ciphertext binding ⇒ worst-of-both-worlds.

5.3 v5: Key Commitment and Anti-Rollback

$$\mathsf{commitment}_{\mathrm{v5}} = \mathsf{HKDF\text{-}Expand}(\mathsf{mk},\ \texttt{"dota-v5-key-commitment"} \,\|\, \mathsf{kdf_params} \,\|\, pk_{\mathsf{KEM}} \,\|\, pk_{\mathsf{DH}}) \tag{12}$$

Upgrade from v4: Introduced key commitment (HKDF-based) and `min_version` anti-rollback floor. Migration metadata tracking.

5.4 v4: HKDF Key Separation

$$K_{\mathsf{KEM}}^{\text{wrap}} = \mathsf{HKDF\text{-}Expand}(\mathsf{mk}, \texttt{"dota-v4-wrap-mlkem"}) \tag{13}$$
$$K_{\mathsf{DH}}^{\text{wrap}} = \mathsf{HKDF\text{-}Expand}(\mathsf{mk}, \texttt{"dota-v4-wrap-x25519"}) \tag{14}$$

Upgrade from v3: Derived separate wrapping keys for ML-KEM and X25519 private keys via purpose-labeled HKDF-Expand, replacing direct master key usage as AES key.

5.5 v3: Structural Refactor

No cryptographic changes. Restructured flat field layout into nested `KemKeyPair` and `X25519KeyPair` structures. Same master-key-as-AES-key wrapping as v2.

5.6 v2: Hybrid KEM Introduction

$$K_{\text{v2}} = \mathsf{HKDF}(\mathsf{ss}_{\mathsf{KEM}} \,\|\, \mathsf{ss}_{\mathsf{DH}},\ \texttt{"dota-v2-hkdf-salt"},\ \texttt{"dota-v2-secret"}) \tag{15}$$

Upgrade from v1: Added Kyber768 (pre-FIPS) as a post-quantum KEM alongside X25519. Introduced hybrid shared secret combination via HKDF.

5.7 v1: X25519-Only Baseline

$$K_{\text{v1}} = \mathsf{DH}(sk_{\mathsf{DH}}, \mathsf{eph}_{\mathsf{pk}}) \quad \text{(used directly as AES key)} \tag{16}$$

No KDF, no KEM, no key separation. The DH shared secret was used directly as the AES-256 encryption key.

6 Migration Security Analysis

The upgrade path vN → v(N+1) is implemented as an in-memory migration chain:

$$\text{v1} \xrightarrow{\text{add KEM}} \text{v2} \xrightarrow{\text{struct}} \text{v3} \xrightarrow{\text{HKDF wrap}} \text{v4} \xrightarrow{\text{commit}} \text{v5} \xrightarrow{\text{FIPS 203}} \text{v6} \xrightarrow{\text{TC-HKEM}} \text{v7}$$

Theorem 6.1 (Migration Security)**.** *The migration chain preserves plaintext confidentiality: no intermediate vault state is written to disk until the complete chain succeeds to v7, and a backup of the original file is created only after successful in-memory migration. Wrong passphrase or corrupted data at any step causes the chain to abort with zero disk writes.*

The v5 → v6 transition is particularly critical: it verifies the v5 HKDF-based key commitment, decrypts under legacy Kyber semantics, generates fresh ML-KEM-768 keypairs, and re-encrypts all secrets. The v6 → v7 transition similarly verifies the v6 HMAC-SHA256 commitment before decrypting and re-encrypting under TC-HKEM with the new passphrase commitment τ.

7 Relation to CPaceOQUAKE+

The CPaceOQUAKE+ protocol [8] (Vos–Jarecki–Wood, CFRG) provides a closely related construction for password-authenticated key exchange. Our TC-HKEM construction draws two key ideas from this work:

1. **Sequential combiner pattern.** CPaceOQUAKE achieves best-of-both-worlds by feeding the CPace session key into OQUAKE's password. Analogously, TC-HKEM feeds mk into the per-secret key derivation via τ.

2. **Transcript binding.** CPaceOQUAKE's key derivation includes the full protocol transcript (all messages, including KEM ciphertexts). TC-HKEM includes $\mathsf{ct}_{\mathsf{KEM}}$ and $\mathsf{eph}_{\mathsf{pk}}$ in the HKDF input for the same reason.

The key difference is that CPaceOQUAKE+ is a *network protocol* (interactive, 5 messages), while TC-HKEM is an *encryption-at-rest* construction (non-interactive, single encapsulation per secret).

8 Implementation

TC-HKEM is implemented in Rust in the DOTA project. Key implementation properties:

- **Zero-copy IKM construction:** The 1216-byte IKM is built in a single pre-allocated buffer and zeroized immediately after HKDF.
- **ZeroizeOnDrop:** All secret types (`MasterKey`, `AesKey`, `MlKemPrivateKey`, `X25519PrivateKey`) implement Rust's `Zeroize` and `ZeroizeOnDrop` traits.
- **Process hardening:** Core dumps disabled (`RLIMIT_CORE`), ptrace blocked (`PR_SET_DUMPABLE`), memory locked (`mlockall`).
- **102/102 tests passing:** Full end-to-end coverage including v1→v7 migration chains with plaintext verification.

9 Conclusion

TC-HKEM provides a provably secure hybrid KEM combiner for password-protected vaults with three simultaneous guarantees: best-of-both-worlds IND-CCA via ciphertext binding (GHP18), passphrase binding via HMAC-SHA256 commitment, and complete domain separation across seven vault format versions. The construction adds less than 1 μs of overhead per secret and is implemented with 102 passing tests in the DOTA post-quantum secrets manager.

References

[1] F. Giacon, F. Heuer, B. Poettering. *KEM Combiners.* PKC 2018. ePrint 2018/024.

[2] NIST. *Module-Lattice-Based Key-Encapsulation Mechanism Standard.* FIPS 203, August 2024.

[3] A. Langley, M. Hamburg, S. Turner. *Elliptic Curves for Security.* RFC 7748, January 2016.

[4] H. Krawczyk, M. Bellare, R. Canetti. *HMAC: Keyed-Hashing for Message Authentication.* RFC 2104.

[5] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF).* RFC 5869, May 2010.

[6] NIST. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode.* SP 800-38D.

[7] A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications.* RFC 9106, September 2021.

[8] J. Vos, S. Jarecki, C. A. Wood. *Hybrid Post-Quantum Password Authenticated Key Exchange.* draft-vos-cfrg-pqpake, CFRG 2025.

[9] A. Arriaga, M. Barbosa, S. Jarecki. *NoIC: PAKE from KEM without Ideal Ciphers.* ePrint 2025/231.

[10] J. Hesse, M. Rosenberg. *PAKE Combiners and Efficient Post-Quantum Instantiations.* ePrint 2024/1621.